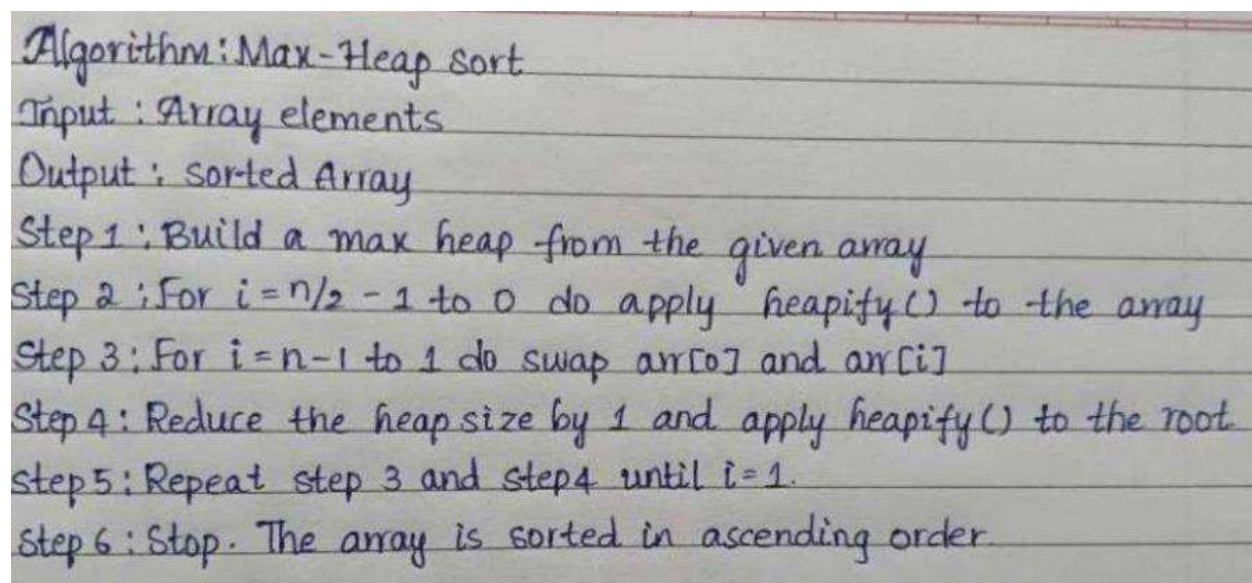


PRACTICAL-3

Aim: Implementation of max heap sort algorithm

Algorithm:



Algorithm: Max-Heap sort
Input : Array elements
Output : sorted Array
Step 1 : Build a max heap from the given array
Step 2 : For $i = n/2 - 1$ to 0 do apply heapify() to the array
Step 3 : For $i = n - 1$ to 1 do swap $arr[0]$ and $arr[i]$
Step 4 : Reduce the heap size by 1 and apply heapify() to the root
Step 5 : Repeat step 3 and step 4 until $i = 1$.
Step 6 : Stop. The array is sorted in ascending order.

Program:

```
#include <iostream>

using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
```

Name :V.Bhavya sri
Enrollment no:92460118227

```
        largest = right;
    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```

int main()

Name :V.Bhavya sri

Enrollment no:92460118227

```
{  
    int arr[] = {12, 11, 13, 5, 6, 7};  
    int n = sizeof(arr) / sizeof(arr[0]);  
    cout << "Original array: ";  
    printArray(arr, n);  
    heapSort(arr, n);  
    cout << "Sorted array: ";  
    printArray(arr, n);  
    return 0;  
}
```

Output:

```
Original array: 12 11 13 5 6 7  
Sorted array: 5 6 7 11 12 13  
  
Process returned 0 (0x0)    execution time : 0.201 s  
Press any key to continue.
```

Time complexity:

Time complexity:

```

for (int i = n/2 - 1; i >= 0; i--) → n/2
    heapify(arr, n, i) → log n } First loop
for (int i = n - 1; i > 0; i--) → n-1
    {
        swap(arr[i], arr[0]) → 1
        heapify(arr, i, 0) → log n } Second loop
    }

```

$T(n) = \frac{n}{2} \log n + (n-1)(1 + \log n)$
 $T(n) = \frac{n}{2} \log n + (n-1) + (n-1) \log n$
 $T(n) = \frac{n}{2} \log n + n \log n + n - 1 - \log n$
 $T(n) = O(n \log n)$

Best case: $O(n \log n)$
 worst case: $O(n \log n)$
 Average case: $O(n \log n)$

Average case:
 Eg: 3, 1, 5, 2, 4
 - For a randomly arranged array, Heapsort builds the heap and repeatedly performs heapify, taking $O(\log n)$ for n elements.
 ∴ Average case = $O(n \log n)$

Worst case: Eg: [5, 4, 3, 2, 1]
 - Even in most unfavourable arrangement, Heapsort still performs heapify operations of $O(\log n)$ for n elements.
 ∴ Worst case = $O(n \log n)$

Conclusion :

The Max-Heap Sort algorithm was successfully implemented in C++. The algorithm first constructs a Max Heap and then repeatedly extracts the maximum element to sort the array in ascending order. Heap Sort has a time complexity of **$O(n \log n)$** in the best, average, and worst cases, making it an efficient sorting algorithm.